



By Nitesh Singh

**Job Ready Free Live Training To Become A**

**SOC**

**Analyst L1 For Free**

Defronix Cyber Security → <https://defronix.com>



**Endpoint Detection and Response (EDR)**, also referred to as endpoint detection and threat response (EDTR), is an endpoint security solution that continuously monitors end-user devices to detect and respond to cyber threats like ransomware and malware.

An EDR operates on a simple hub-and-spoke model. Light agents sit directly on endpoints (laptops, servers, cloud VMs) and stream telemetry back to a centralized cloud engine for heavy-duty analysis.

Below are some of the EDR solutions in the market:

- **CrowdStrike Falcon**
- **SentinelOne ActiveEDR**
- **Microsoft Defender for Endpoint**
- **OpenEDR\**
- **Symantec EDR**

Several other EDR solutions are available in the market. Their underlying architecture is mostly similar, but the features may vary.



[**VISIBILITY**] —> Records a PDF opening and launching a hidden background script.

[**DETECTION**] —> Analyzes the script behavior, recognizes it as credential theft, fires an alert.

[**RESPONSE**] —> Isolates the laptop from the network and kills the script automatically.

# How EDR Works (The Operational Flow)



An EDR does not sleep. It constantly loops through a four-stage lifecycle to keep the environment secure:

## Stage 1: Continuous Data Collection (Telemetry)

The endpoint agent records every fundamental action taking place on the operating system. This data stream includes:

- **Process Executions:** What applications are launching? (e.g., Is `cmd.exe` being opened by `word.exe`?)
- **Registry Changes:** Is a program trying to modify boot settings to stay on the system after a reboot?
- **Network Connections:** What internal or external IP addresses is this specific machine talking to?
- **File Modifications:** Are thousands of documents suddenly changing extensions to `.locked`? (Ransomware behavior).

## Stage 2: Behavioral Analysis & Detection

The cloud console ingests this massive lake of telemetry. Instead of just looking for static file signatures, it runs the data against advanced behavioral models, machine learning, and threat intelligence feeds.

**The Secret Weapon: Behavioral Heuristics.** An EDR doesn't care if a script has a completely unique name it has never seen before. If that script attempts to dump the OS password database (LSASS memory), the EDR flags it based on *what it is doing*, not what it is named.



### Stage 3: Investigation & Threat Hunting

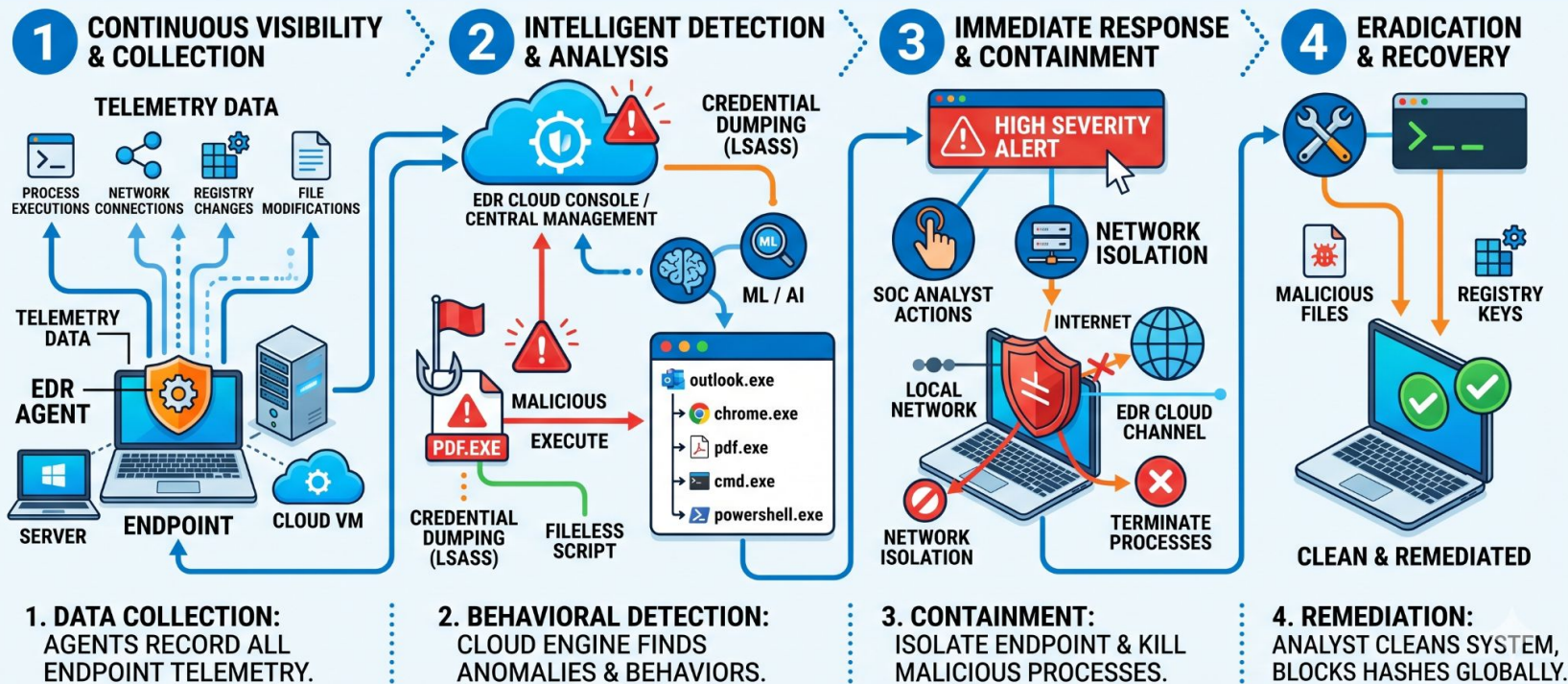
When the detection engine spots an anomaly, it packages it into an alert for analysts like us. Because the EDR records everything, it gives us a historical timeline. We can trace an attack back to its root cause (Patient Zero) to see exactly how the hacker got in—whether they downloaded a malicious email attachment or exploited an unpatched application.

### Stage 4: Automated & Manual Response

This is where the "R" in EDR earns its keep. If the EDR detects an active, high-speed threat like ransomware, it doesn't wait for a human analyst to wake up. It can execute automated containment actions instantly.



# EDR (ENDPOINT DETECTION AND RESPONSE) WORKFLOW: REAL-WORLD MALWARE ATTACK & RESPONSE



## EDR vs. Traditional Antivirus



| Feature          | Traditional Antivirus (AV)                                     | Endpoint Detection & Response (EDR)                |
|------------------|----------------------------------------------------------------|----------------------------------------------------|
| Detection Method | Signatures (Knows what bad files <i>look</i> like)             | Behaviors (Knows what bad actors <i>do</i> )       |
| Data Retention   | Only logs when something is blocked                            | Logs everything continuously (Even benign data)    |
| Visibility       | Blind to sophisticated fileless or living-off-the-land attacks | Full visibility into memory injections and scripts |
| Response Options | Delete or quarantine a single file                             | Isolate hosts, terminate processes, remote shell   |

## Feature-by-Feature Comparison Table



| Feature                 | Traditional Antivirus (AV)                                                                                | Endpoint Detection & Response (EDR)                                                   |
|-------------------------|-----------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| <b>Primary Goal</b>     | <b>Prevention:</b> Stop known malware from executing.                                                     | <b>Detection &amp; Response:</b> Spot active threats and provide tools to fight them. |
| <b>Data Logging</b>     | <b>Passive:</b> Only logs an event when a known virus is blocked.                                         | <b>Continuous:</b> Logs all system activity (processes, network, registry) 24/7.      |
| <b>Response Actions</b> | Quarantines or deletes a single file.                                                                     | Isolate hosts from the network, kill live processes, remote shell.                    |
| <b>Fileless Attacks</b> | <b>Blind:</b> Cannot detect attacks running purely in system memory (e.g., malicious PowerShell scripts). | <b>Full Visibility:</b> Monitors volatile memory spaces and active script executions. |
| <b>Historical Data</b>  | None. If an incident happens, you cannot trace <i>how</i> the malware got there.                          | <b>Flight Recorder:</b> Provides a full timeline of events leading up to the attack.  |

## Analyzing the EDR Process Tree



The SIEM has just triggered a high-severity alert: "**Suspicious PowerShell Execution Detected.**"

We need to figure out how this happened, what the attacker did, and how to stop them before they move laterally (hop to other machines on the network).

To do this, we open our EDR console and pull up the **Process Tree** for the compromised asset. The EDR gives us the "family tree" of the execution, showing us exactly who spawned whom.

```
[1] outlook.exe (Email Client)
├── [2] chrome.exe (User clicked a link, opened browser)
│   └── [3] invoice_9932.pdf.exe (Downloaded file masquerading as a
PDF)
│       └── [4] cmd.exe (Spawns command line wrapper)
│           └── [5] powershell.exe -ExecutionPolicy Bypass -Enc
WwB... (Malicious script)
```



## The Analyst's Breakdown:

1. **Outlook & Chrome:** The user received an email, clicked a link, and downloaded a file.
2. **The "Double Extension" Trick:** Look closely at step [ 3 ]. The file is named `invoice_9932.pdf.exe`. Windows often hides the `.exe` extension by default, so the user thought they were opening a regular PDF. This is classic **Phishing**.
3. **The PowerShell Command:** The command contains flags like `-ExecutionPolicy Bypass` (tells Windows to ignore security restrictions on scripts) and `-Enc` (stands for **Encoded**, meaning the attacker obfuscated the script into a giant string of random gibberish like `WwB . . .` to hide what it does from basic text filters).

## Decoding the Payload & Investigating Behavior



Using our EDR tools, we decode the obfuscated script. We see the script trying to execute two specific actions on the host operating system:

- **Registry Modification:** It writes a key to `HKCU\Software\Microsoft\Windows\CurrentVersion\Run`. This is a persistence mechanism—it ensures that even if the user reboots their laptop, the malware launches again automatically.
- **LSASS Dumping:** The script attempts to read the memory space of `lsass.exe` (Local Security Authority Subsystem Service). **Red Alert:** This is where Windows caches user passwords and tokens in memory. The attacker is trying to steal credentials to hijack other identities.

## Taking Immediate Response Action



The attacker has active code execution and is trying to harvest passwords. We cannot afford to wait. We execute our EDR containment playbook directly from the console:

### 1. Network Isolation: Takes 2 seconds.

We hit the **Isolate Host** button in the EDR. The laptop is instantly cut off from the local office network and the internet. The attacker's command-and-control (C2) server connection drops. They can no longer send commands or pull stolen data out.

### 2. Process Termination: Takes 5 seconds.

We issue a command through the EDR to kill the running powershell.exe and cmd.exe processes, completely halting the active malicious script.

### 3. Remediation & Cleanup: Takes 10 minutes.

Using the **Live Response Remote Shell**, we securely connect to the isolated machine. We delete the downloaded invoice\_9932.pdf.exe file and scrub the persistence registry key the script created.

## Closing the Loop (Post-Incident)



Now that the asset is safe, we don't just stop there. We pivot to the other concepts we learned earlier to ensure total eradication:

- **Identity Check:** We pivot to our **Identity** tool (e.g., Microsoft Entra ID) and force a password reset and session revocation for this user, just in case the attacker managed to scrape their credentials before we killed the process.
- **Workbook Lookup:** We create a **Lookup** containing the malicious file hash and the IP address the script tried to talk to. We run a global query across our SOC **Workbook** to see if any other machine in the entire enterprise has downloaded the same file.

This is how an EDR transforms a potential company-wide disaster into a 15-minute contained incident.

Within a network, you will see Firewalls, DLPs, Email Security Gateways, IAMs, EDRs, and other security solutions protecting the different components of the network. To minimize the effort and maximize the efficiency, all these security solutions are integrated with a **SIEM (Security Information and Event Management)** solution that becomes the central point of investigation for the analysts.